P2

Anatomía ELF/PE y direcciones — Criterios y solución docente

Parte I · RA2 · 2 h

Universidad de Almería · Edición de 27 de septiembre de 2026

Documento: Criterios y solución de referencia
Asignatura: Ingeniería Inversa y Análisis Forense Digital
Uso: Laboratorio docente en entorno controlado y sobre las muestras indicadas
Alumno/a: _____ Grupo: _____

Material reservado para corrección docente

Este documento contiene criterios de corrección y resultados de referencia. No debe incorporarse al paquete evaluable que recibe el alumnado.

Índice

1. Solución y criterios **2**

2. Desarrollo de referencia **2**
2.1. ELF 2
2.2. PE 2

3. Ejemplo de corrección de una conversión **3**

4. Lista de corrección **3**

5. Solución PE-01 1.0 — 28 de septiembre de 2026 **3**
5.1. Conversión numérica y bytes comprobados 4
5.2. Memoria sin bytes y dependencias 4

> **Estado actual: PE-01 1.0 disponible.** La actualización del 28 de septiembre de 2026, al final del documento, contiene el PE manifestado y sus datos comprobados. Sustituye las referencias anteriores a su preparación pendiente; se conserva la ficha histórica.

Material para el profesorado. No incluir en el paquete evaluable del alumnado. No se tratan como secretos los fuentes o soluciones publicados en el repositorio.

1 Solución y criterios

Los valores numéricos dependen del build; se corrige usando las cabeceras del fichero cuyo SHA-256 figura en la entrega. `e_entry` suele señalar arranque de runtime, no `main`. Un `ET_DYN` se interpreta junto al resto de metadatos para identificar PIE.

ELF: `offset=p_offset+(VA-p_vaddr)` solo dentro del rango `p_filesz` del segmento; VA de ejecución añade load bias. PE: `VA=base+RVA`; para bytes de una sección, `offset=PointerToRawData+RVA-VirtualAddress`, validando `SizeOfRawData`. Se exige comprobación de bytes, no solo escribir fórmulas. Un `AddressOfEntryPoint` no se trata como VA absoluta. `.bss` no tiene el contenido cero almacenado en disco.

No exigir una lista fija de imports (cambia con compilador, biblioteca y optimización), ni la presencia de `.plt.sec`. La tabla dinámica no contiene exclusivamente imports; stripped tampoco implica ausencia de cualquier nombre. La fecha PE no se autentica por aparecer en la cabecera. Si falta el PE, el docente debe suministrarlo antes de la sesión: no se omite RA2 ni se instala Windows durante la práctica.

Rúbrica y pesos: evaluación de Parte I.

2 Desarrollo de referencia

2.1 ELF

La muestra local validada es ELF64, little endian, x86-64, dinámica y PIE (`ET_DYN`), pero los valores se vuelven a leer del build distribuido. Para una VA:

1. localizar el `PT_LOAD` cuyo intervalo contiene la VA;
2. comprobar que cae en `[p_vaddr,p_vaddr+p_filesz)` si se esperan bytes;
3. calcular `p_offset+(VA-p_vaddr)`;
4. comparar bytes con `xxd` y desensamblado.

Una dirección comprendida solo en la cola hasta `p_memsz` pertenece a memoria inicializada por el cargador y no admite esa comprobación de bytes del fichero. `.bss` es el ejemplo habitual.

En el build local de control, el entry point es 0x1150. Cae en el `PT_LOAD` ejecutable con `p_offset=0x1000`, `p_vaddr=0x1000` y `p_filesz=0x2d9`:

```
offset = 0x1000 + (0x1150 - 0x1000) = 0x1150
```

`xxd` en 0x1150 comienza por `f3 0f 1e fa 31 ed 49 89 ...`, bytes que también muestra el desensamblado del entry point. Es un ejemplo comprobado de esta compilación, no una respuesta para otro hash. En el mismo build `.bss` es `NOBITS`, reforzando que su dirección en memoria no representa contenido almacenado como una sección de bytes.

2.2 PE

El `AddressOfEntryPoint` se trata como RVA. La VA preferida suma `ImageBase`. Para el offset se selecciona la sección que contiene el RVA y se usa su `PointerToRawData`. Se rechaza una respuesta que sume directamente base y RVA para buscar ese número como offset.

El PE no estaba presente en la validación local más reciente; por ello la solución no publica números inventados. El docente debe generarlo con MinGW-w64 o entregar el fichero manifestado antes de evaluar esa mitad.

3 Ejemplo de corrección de una conversión

Se da por completa cuando aparecen valores de cabecera, sustitución en la fórmula, validación del rango y bytes observados. Una fórmula correcta con un segmento/sección incorrectos no acredita el resultado.

4 Lista de corrección

- Hash y formato de ambos ficheros.
- Segmentos frente a secciones.
- Entry point frente a `main`.
- Conversión ELF y PE comprobadas.
- Caso sin respaldo en fichero explicado.
- Imports definidos/indefinidos correctamente.
- Límites de ASLR, timestamps y dependencias.

5 Solución PE-01 1.0 — 28 de septiembre de 2026

El PE ya está generado. Esta sección sustituye el estado de ausencia documentado arriba, conservado como antecedente. La edición distribuida ocupa 39.936 bytes y está identificada por el manifiesto. SHA-256:

```
ed7640c92c67e0308d8a7c2edc21f3d7c25f2c435a8ed17cde23cdcd0eac5015
```

Las cabeceras capturadas y resultados completos proceden del fichero real. El parser Python de cabeceras y GNU objdump se han contrastado.

Campo	Valor observado	Interpretación
Machine	0x8664	AMD64
Magic	0x20b	PE32+
Subsystem	3	consola Windows
ImageBase	0x140000000	base preferida
AddressOfEntryPoint	0x1410	RVA, no offset
SectionAlignment	0x1000	alineamiento virtual
FileAlignment	0x200	alineamiento raw
DllCharacteristics	0x160	HIGH_ENTROPY_VA, DYNAMIC_BASE y NX_COMPAT
Timestamp COFF	0	fijado por el build; no fecha histórica
Tabla COFF de símbolos	0 entradas	copia stripped; los imports permanecen

5.1 Conversión numérica y bytes comprobados

El entry point está en .text: RVA inicial 0x1000, VirtualSize=0x6c88, RawPtr=0x400 y RawSize=0x6e00. Por tanto:

```
VA preferida = 0x140000000 + 0x1410 = 0x140001410
delta = 0x1410 - 0x1000 = 0x410
delta + 16 = 0x420 < 0x6c88 y < 0x6e00
offset = 0x400 + 0x410 = 0x810
```

```
xxd -g 1 -s 0x810 -l 16 ejemplos-pe.exe
objdump -d -M intel --start-address=0x140001410 \
  --stop-address=0x140001420 ejemplos-pe.exe
```

Bytes observados:

```
55 48 89 e5 48 83 ec 20 48 8b 05 c1 83 00 00 c7
```

La ventana de 16 bytes termina dentro de una instrucción: no significa que todas las instrucciones del intervalo estén completas. El desensamblado docente identifica mainCRTStartup en VA preferida 0x140001410 y main en 0x140007ba0. Son direcciones diferentes: el entry point entra en el runtime. Tras strip se conservan RVA y bytes de las secciones retenidas, pero pueden cambiar sus offsets de fichero.

5.2 Memoria sin bytes y dependencias

.bss comienza en RVA 0xc000, tiene VirtualSize=0xb80 y RawSize=0. Su primer byte virtual correspondería a VA preferida 0x14000c000 y no tiene un byte de variable guardado en el fichero. Aplicar mecánicamente RawPtr+delta daría cero: leer MZ allí sería confundir una cabecera con memoria inicializada por el cargador.

La muestra tiene diez secciones: .text, .data, .rdata, .pdata, .xdata, .bss, .idata, .CRT, .tls y .reloc. Importa KERNEL32.dll y msvcrt.dll; el detalle de funciones se conserva en inspeccion.json. Buena parte del código y los imports proceden del runtime. Su presencia no demuestra acciones históricas de la aplicación.

La aplicación propia realiza aritmética, imprime resultados y compara una cadena; no implementa red, persistencia ni modificación de documentos. Se ha revisado el fuente y verificado el formato sin ejecutar el PE. No se afirma una sesión Windows, una base ASLR observada ni una fecha real a partir del timestamp cero.